

POLITECHNIKA LUBELSKA
SYSTEMY SZTUCZNEJ INTELIGENCJI

System predykcji wyników wyścigów
Formuły 1 z rozmytym systemem
ekspertskim wspomagającym decyzje
strategiczne o pit-stopach

Autor:

Hubert Kiszka

Prowadzący:

dr inż. Tomasz Nowicki

Lublin, 31 maja 2026

1 Wstęp i opis problemu

Projekt podejmuje temat modelowania i wspomagania decyzji w wyścigach Formuły 1. Głównym celem systemu jest rozwiązanie dwóch powiązanych ze sobą problemów decyzyjnych:

1. **Predykcja wyniku wyścigu:** Prognozowanie, do której z trzech kategorii końcowych (*Podium*, *Punkty*, *Poza punktami*) trafi dany kierowca, bazując na danych historycznych zawodnika na danym torze, pozycji startowej oraz bieżącej formie zespołu i zawodnika.
2. **Strategia zjazdów do boksu:** Opracowanie systemu eksperckiego z przetwarzeniem rozmytym, określającego pilność wykonania pit-stopu na podstawie stopnia zużycia opon, etapu wyścigu, dystansu do rywala oraz prognozy szans na bycie w punktach lub na podium dostarczonej przez system ekspercki.

Do realizacji projektu wykorzystano relacyjny zbiór danych *Formula 1 World Championship (1950-2024)* dostępny na platformie Kaggle.

2 Przetwarzanie danych i inżynieria cech

Surowe dane składają się z wielu powiązanych tabel. W celu przygotowania zbioru treningowego dokonano operacji złączenia kluczowych tabel: `results`, `races`, `drivers`, `constructors`, `qualifying`, `driver_standings` oraz `constructor_standings`.

2.1 Czyszczenie danych i transformacja

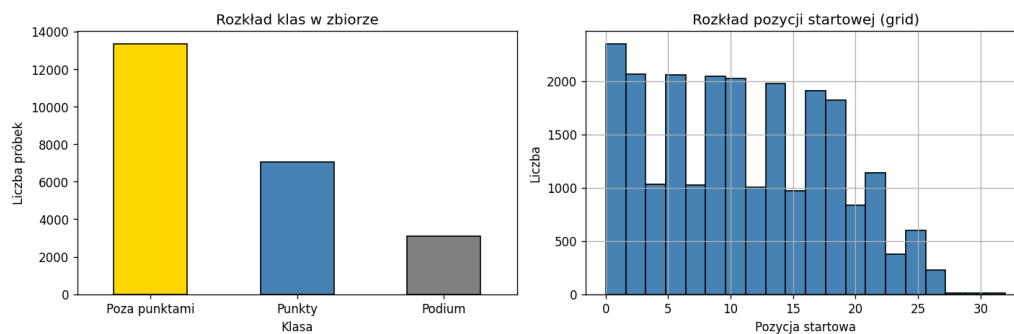
Wartości brakujące, reprezentowane jako `\N`, zostały zastąpione wartościami `NaN`. Problem przewidywania pozycji końcowej przekształcono z regresji w zadanie klasyfikacji wieloklasowej poprzez zdefiniowanie zmiennej celu `target`:

$$\text{target} = \begin{cases} \text{Podium} & \text{gdy } \text{positionOrder} \leq 3 \\ \text{Punkty} & \text{gdy } 4 \leq \text{positionOrder} \leq 10 \\ \text{Poza punktami} & \text{gdy } \text{positionOrder} > 10 \end{cases} \quad (1)$$

2.2 Inżynieria cech

Wskaźniki formy zawodników i zespołów obliczano dynamicznie z wykorzystaniem średniej skumulowanej oraz przesunięcia aby obliczać zawsze wyniki tylko do poprzedniego wyniku (Jeżeli model ma określić wynik kierowcy w wyścigu w rundzie 4, to powinien tylko znać wyniki z rund 1,2 oraz 3) . Kluczowe cechy wejściowe modelu (**FEATURES**) obejmują:

- `grid`, `grid_quali` – pozycje startowa oraz kwalifikacyjna,
- `avg_grid_season`, `avg_position_season` – dotychczasowa średnia pozycja startowa i końcowa w bieżącym sezonie,
- `avg_constructor_season` – średnia pozycja końcowa konstruktora,
- `driver_pos_before_race`, `constructor_pos_before_race` – pozycja w mistrzostwach przed wyścigiem,
- `hist_track_avg` – historyczna średnia pozycja kierowcy na danym torze.



Rysunek 1: Rozkład klas decyzyjnych oraz rozkład pozycji startowych w zbiorze.

Cechy numeryczne poddano standaryzacji (`StandardScaler`). Zbiór podzielono na część treningową (80%) i testową (20%).

3 Modelowanie i architektura systemu

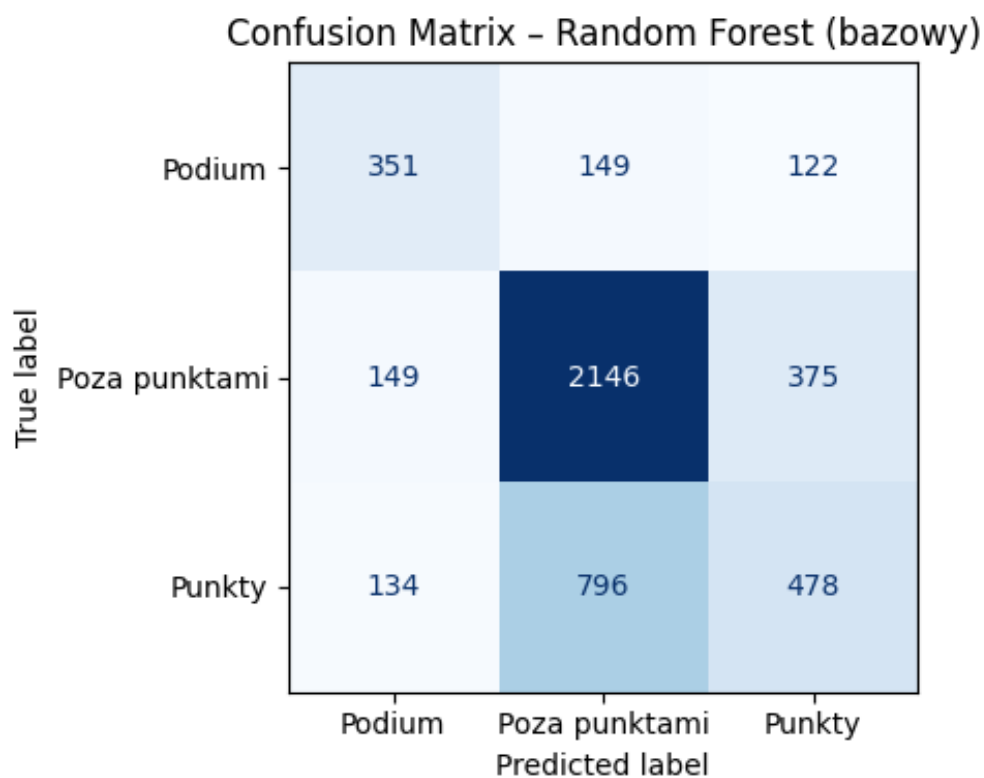
3.1 Random Forest - model bazowy

=== Random Forest (bazowy) ===

Accuracy: 0.6330

F1 (weighted): 0.6174

	precision	recall	f1-score	support
Podium	0.55	0.56	0.56	622
Poza punktami	0.69	0.80	0.75	2670
Punkty	0.49	0.34	0.40	1408
accuracy			0.63	4700
macro avg	0.58	0.57	0.57	4700
weighted avg	0.61	0.63	0.62	4700



Rysunek 2: Metryki Random Forest oraz macierz pomyłek.

3.2 Random Forest - Kalibracja GridSearchCV

Fitting 5 folds for each of 27 candidates, totalling 135 fits

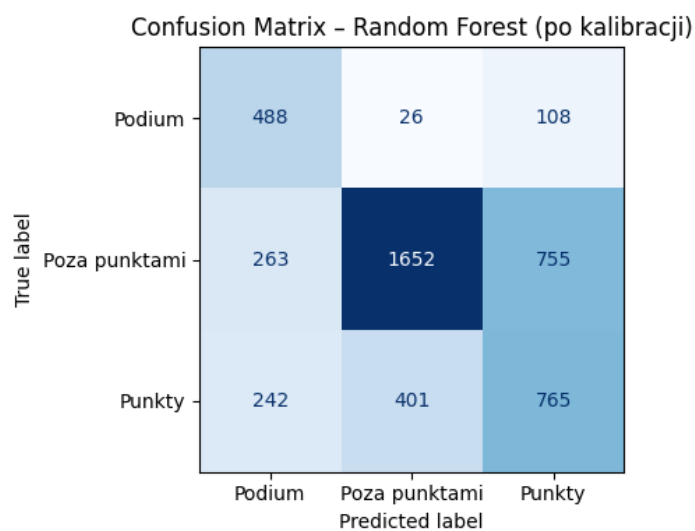
Najlepsze parametry RF: {'max_depth': 10, 'min_samples_split': 2, 'n_estimators': 100}

=== Random Forest (po kalibracji) ===

Accuracy: 0.6181

F1 (weighted): 0.6262

	precision	recall	f1-score	support
Podium	0.49	0.78	0.60	622
Poza punktami	0.79	0.62	0.70	2670
Punkty	0.47	0.54	0.50	1408
accuracy			0.62	4700
macro avg	0.59	0.65	0.60	4700
weighted avg	0.66	0.62	0.63	4700



Rysunek 3: Metryki skalibrowanego Random Forest oraz macierz pomyłek.

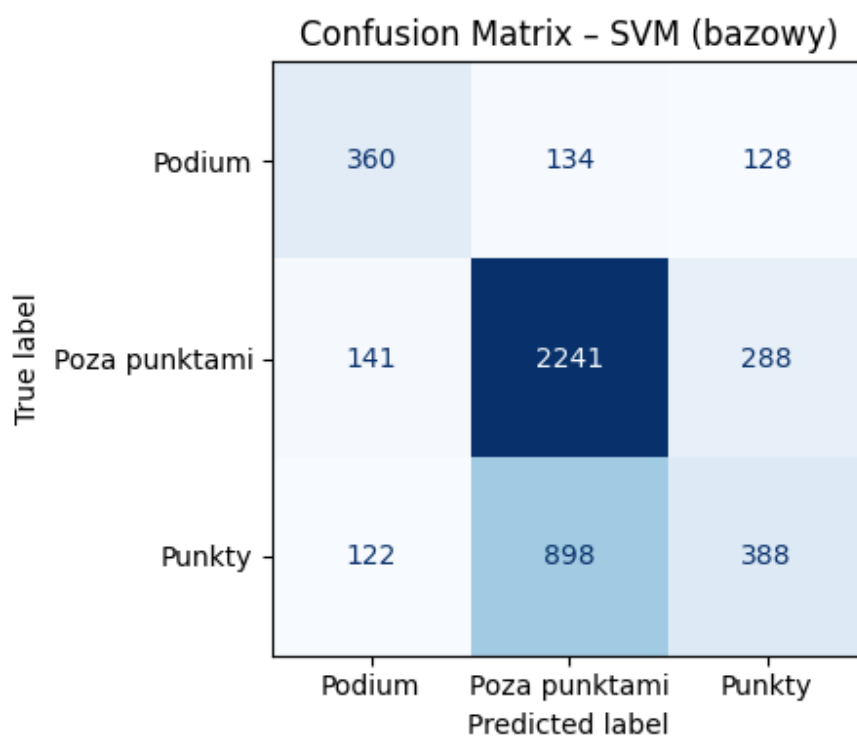
3.3 Support Vector Machine - model bazowy

```

=== SVM (bazowy) ===
Accuracy:      0.6360
F1 (weighted): 0.6101

```

	precision	recall	f1-score	support
Podium	0.58	0.58	0.58	622
Poza punktami	0.68	0.84	0.75	2670
Punkty	0.48	0.28	0.35	1408
accuracy			0.64	4700
macro avg	0.58	0.56	0.56	4700
weighted avg	0.61	0.64	0.61	4700



Rysunek 4: Metryki SVM oraz macierz pomyłek.

3.4 Support Vector Machine - Kalibracja GridSearchCV

Fitting 5 folds for each of 6 candidates, totalling 30 fits

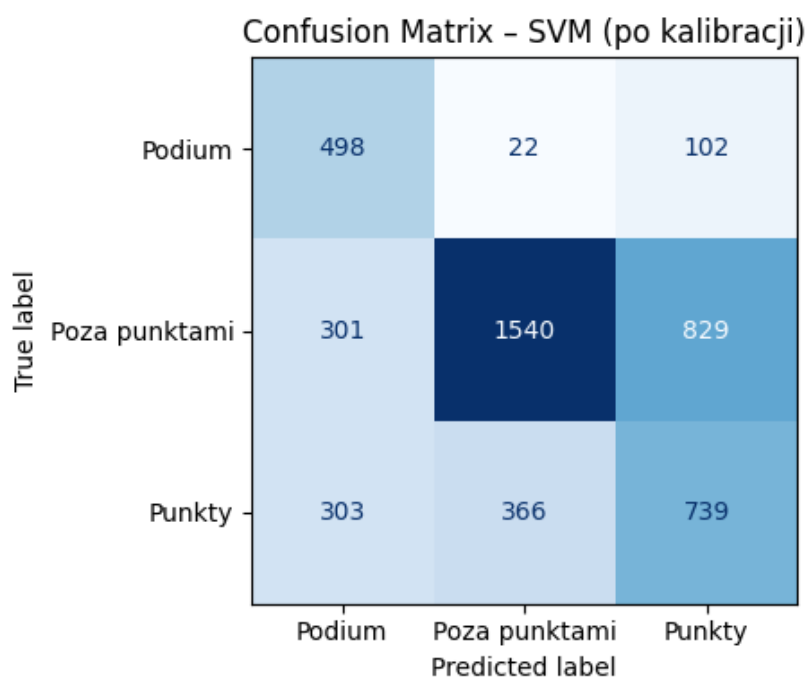
Najlepsze parametry SVM: {'C': 10, 'gamma': 0.01, 'kernel': 'rbf'}

=== SVM (po kalibracji) ===

Accuracy: 0.5909

F1 (weighted): 0.6008

	precision	recall	f1-score	support
Podium	0.45	0.80	0.58	622
Poza punktami	0.80	0.58	0.67	2670
Punkty	0.44	0.52	0.48	1408
accuracy			0.59	4700
macro avg	0.56	0.63	0.58	4700
weighted avg	0.65	0.59	0.60	4700

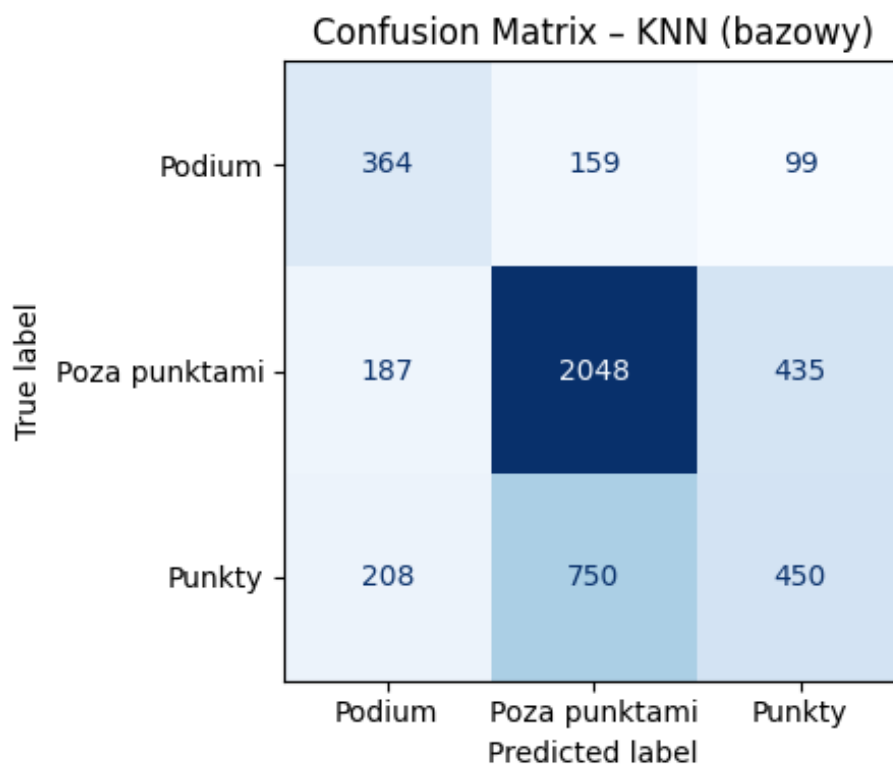


Rysunek 5: Metryki skalibrowanego SVM oraz macierz pomyłek.

3.5 K Nearest Neighbors - model bazowy

```
=== KNN (bazowy) ===
Accuracy:      0.6089
F1 (weighted): 0.5960
```

	precision	recall	f1-score	support
Podium	0.48	0.59	0.53	622
Poza punktami	0.69	0.77	0.73	2670
Punkty	0.46	0.32	0.38	1408
accuracy			0.61	4700
macro avg	0.54	0.56	0.54	4700
weighted avg	0.59	0.61	0.60	4700



Rysunek 6: Metryki KNN oraz macierz pomyłek.

3.6 K Nearest Neighbors - Kalibracja GridSearchCV

Fitting 5 folds for each of 30 candidates, totalling 150 fits

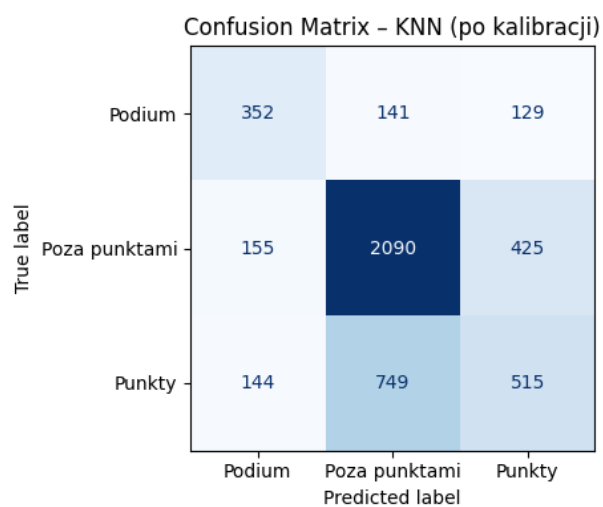
Najlepsze parametry KNN: {'metric': 'manhattan', 'n_neighbors': 15, 'weights': 'distance'}

=== KNN (po kalibracji) ===

Accuracy: 0.6291

F1 (weighted): 0.6180

	precision	recall	f1-score	support
Podium	0.54	0.57	0.55	622
Poza punktami	0.70	0.78	0.74	2670
Punkty	0.48	0.37	0.42	1408
accuracy			0.63	4700
macro avg	0.57	0.57	0.57	4700
weighted avg	0.61	0.63	0.62	4700



Rysunek 7: Metryki skalibrowanego KNN oraz macierz pomyłek.

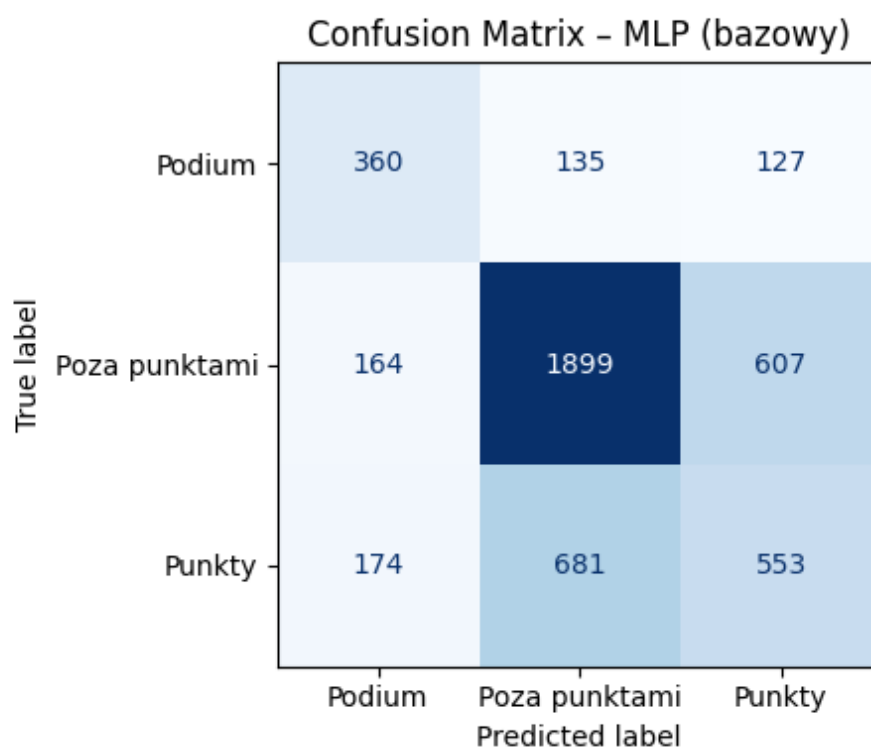
3.7 MLP - model bazowy

```

=== MLP (bazowy) ===
Accuracy:      0.5983
F1 (weighted): 0.5958

```

	precision	recall	f1-score	support
Podium	0.52	0.58	0.55	622
Poza punktami	0.70	0.71	0.71	2670
Punkty	0.43	0.39	0.41	1408
accuracy			0.60	4700
macro avg	0.55	0.56	0.55	4700
weighted avg	0.59	0.60	0.60	4700



Rysunek 8: Metryki MLP oraz macierz pomyłek.

3.8 MLP - Kalibracja GridSearchCV

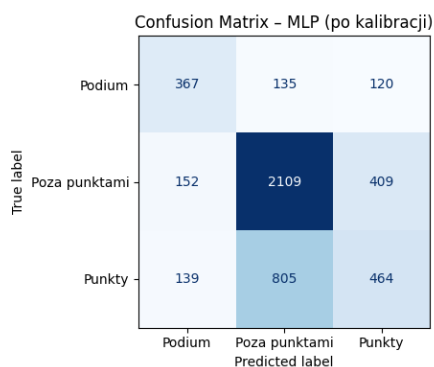
```
Fitting 5 folds for each of 24 candidates, totalling 120 fits

Najlepsze parametry MLP: {'activation': 'relu', 'alpha': 0.001, 'hidden_layer_sizes': (64, 32), 'learning_rate_init': 0.001}

=== MLP (po kalibracji) ===
Accuracy:      0.6255
F1 (weighted): 0.6107
precision      recall    f1-score   support

   Podium      0.56      0.59      0.57      622
 Poza punktami 0.69      0.79      0.74     2670
   Punkty      0.47      0.33      0.39     1408

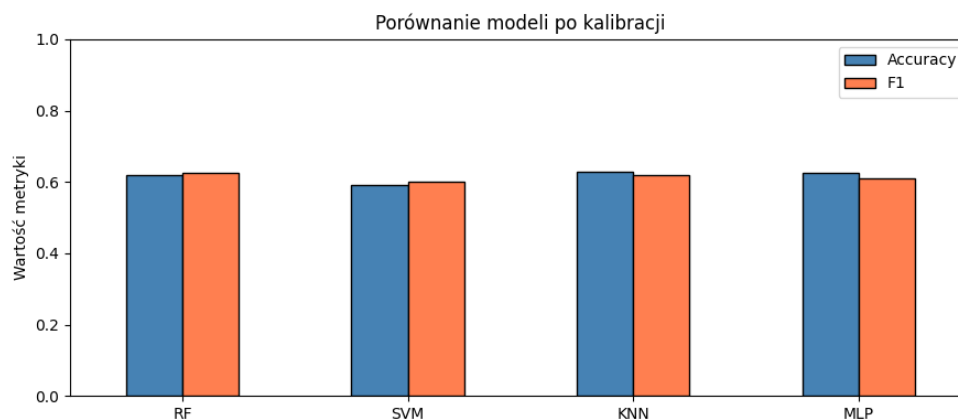
 accuracy      0.63      4700
 macro avg     0.57      0.57      4700
 weighted avg  0.61      0.63      4700
```



Rysunek 9: Metryki skalibrowanego MLP oraz macierz pomyłek.

3.9 Porównanie modeli po kalibracji

	Accuracy	F1
RF	0.6181	0.6262
SVM	0.5909	0.6008
KNN	0.6291	0.6180
MLP	0.6255	0.6107



Rysunek 10: Porównanie Accuracy i F1 score modeli.

3.10 System ekspercki - VotingClassifier

Zaimplementowany system ekspercki oparty na VotingClassifier wykorzystał soft voting gdzie każdy model wylicza prawdopodobieństwo. Osiągnął ostatecznie ogólną dokładność

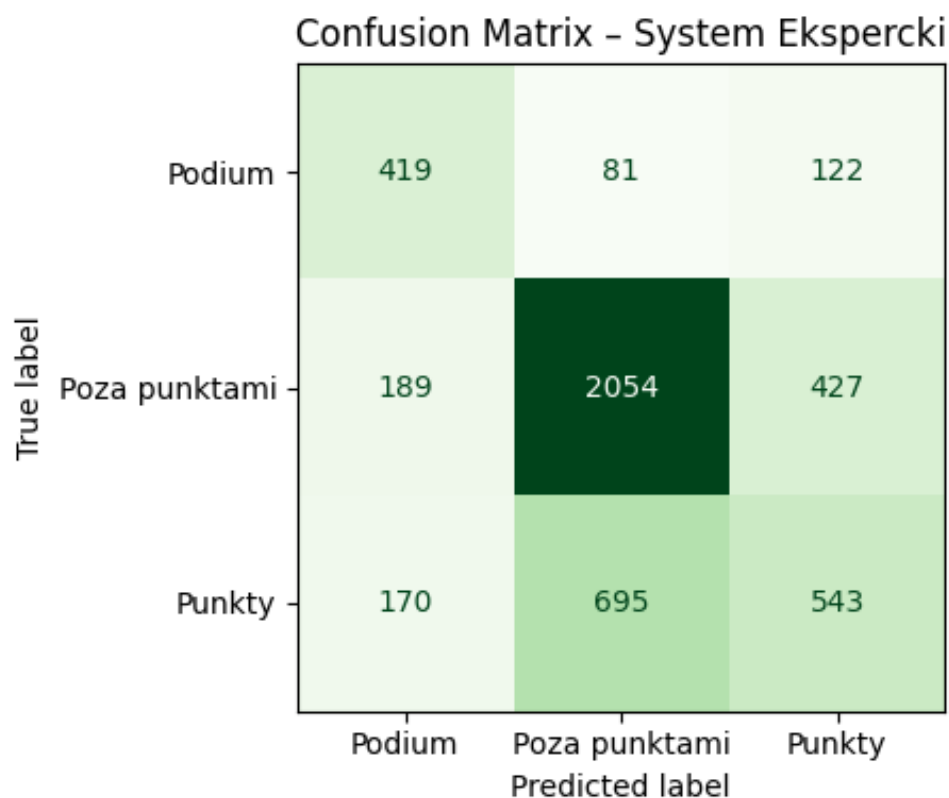
na poziomie **Accuracy = 64.26%**.

```

=== System Ekspercki (VotingClassifier) ===
Accuracy:      0.6417
F1 (weighted): 0.6337

```

	precision	recall	f1-score	support
Podium	0.54	0.67	0.60	622
Poza punktami	0.73	0.77	0.75	2670
Punkty	0.50	0.39	0.43	1408
accuracy			0.64	4700
macro avg	0.59	0.61	0.59	4700
weighted avg	0.63	0.64	0.63	4700



Rysunek 11: Porównanie Accuracy i F1 score modeli.

3.11 Logika rozmyta - doradca pit-stopów

Wynik z modelu jest przekazywany do systemu wnioskowania rozmytego typu Mamdaniego. Modeluje on strategię pit-stopów na podstawie:

- **Wejścia:** zuzycie opon [0-100%], przewaga [-10s do 10s], numer_okrazenia oraz

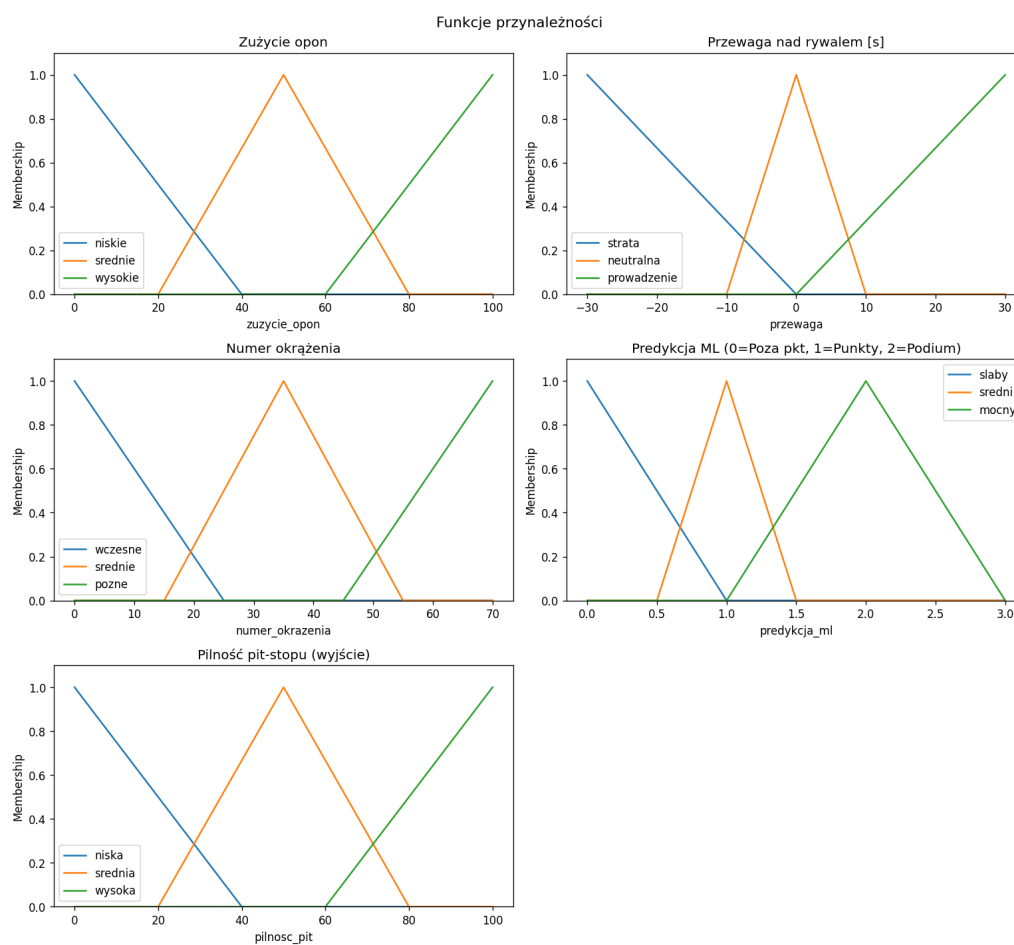
predykcja_ml (prognoza szans na zajęcie pozycji na podium/w punktach/poza punktami).

- **Wyjście:** pilnosc_pit [0-100] – stopień krytyczności zmiany opon.

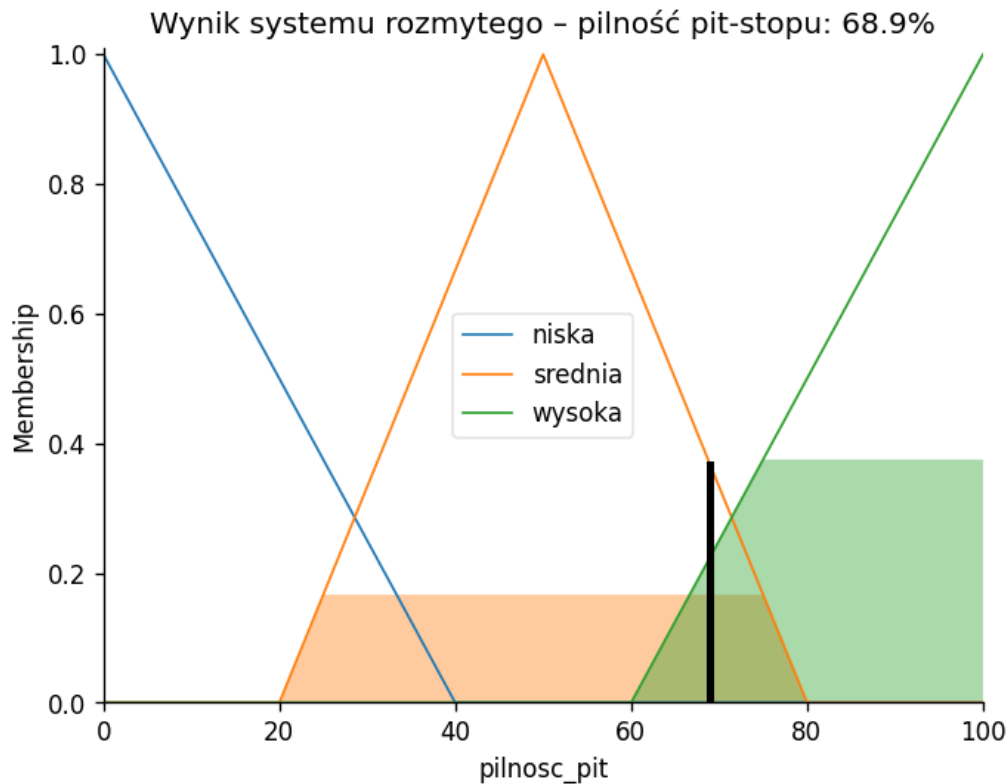
Baza reguł uwzględnia np. bardzo wysokie zużycie opon przy spadającym tempie natychmiast generuje komunikat **[PIT]**, podczas gdy dobre tempo odradza decyzję do komunikatu **[ROZWAŻ PIT]** lub **[ZOSTAŃ]**.

```
rules = [  
    # Reguła 1: Opony bardzo zużyte -> pit  
    ctrl.Rule(zuzycie_opon["wysokie"], pilnosc_pit["wysoka"]),  
    # Reguła 2: Opony świeże + 1-miejsce -> zostań  
    ctrl.Rule(zuzycie_opon["niskie"] & przewaga["prowadzenie"], pilnosc_pit["niska"]),  
    # Reguła 3: Środkowe okrążenia + średnie zużycie -> być może pit  
    ctrl.Rule(numer_okrazenia["srednie"] & zuzycie_opon["srednie"], pilnosc_pit["srednia"]),  
    # Reguła 4: Poza punktami + późne okrążenia -> pit  
    ctrl.Rule(przewaga["strata"] & numer_okrazenia["pozne"], pilnosc_pit["wysoka"]),  
    # Reguła 5: Wczesne okrążenia -> nie pit  
    ctrl.Rule(numer_okrazenia["wczesne"], pilnosc_pit["niska"]),  
    # Reguła 6: Średnie zużycie + strata -> rozważ pit  
    ctrl.Rule(zuzycie_opon["srednie"] & przewaga["strata"], pilnosc_pit["srednia"]),  
    # Reguła 7: Prowadzenie + późne okrążenia + średnie zużycie -> być może pit  
    ctrl.Rule(przewaga["prowadzenie"] & numer_okrazenia["pozne"] & zuzycie_opon["srednie"], pilnosc_pit["srednia"]),  
  
    # Reguła 8: Modele przewidują słaby wynik + średnie zużycie -> pit  
    ctrl.Rule(predykcja_ml["slaby"] & zuzycie_opon["srednie"], pilnosc_pit["wysoka"]),  
    # Reguła 9: Modele przewidują podium + prowadzenie -> nie pit  
    ctrl.Rule(predykcja_ml["mocny"] & przewaga["prowadzenie"], pilnosc_pit["niska"]),  
    # Reguła 10: Modele przewidują podium + średnie zużycie -> być może pit  
    ctrl.Rule(predykcja_ml["mocny"] & zuzycie_opon["srednie"], pilnosc_pit["niska"]),  
    # Reguła 11: Modele niepewne (Punkty) + wysokie zużycie -> pit  
    ctrl.Rule(predykcja_ml["sredni"] & zuzycie_opon["wysokie"], pilnosc_pit["wysoka"]),  
    # Reguła 12: Modele niepewne (Punkty) + niskie zużycie -> nie pit  
    ctrl.Rule(predykcja_ml["sredni"] & zuzycie_opon["niskie"], pilnosc_pit["niska"]),  
]
```

Rysunek 12: Reguły logiczne.



Rysunek 13: Funkcje przynależności.



Rysunek 14: Wynik systemu, **zużycie opon: 75%, przewaga: 5s, numer okrążenia: 40/70**

4 Kod

```

1      import pandas as pd
2      import numpy as np
3      import matplotlib.pyplot as plt
4      from sklearn.model_selection import
5          train_test_split, GridSearchCV, cross_val_score
6      from sklearn.preprocessing import StandardScaler
7      from sklearn.ensemble import RandomForestClassifier
8          , VotingClassifier, IsolationForest
9      from sklearn.svm import SVC
10     from sklearn.neighbors import KNeighborsClassifier
11     from sklearn.neural_network import MLPClassifier
12     from sklearn.metrics import accuracy_score,
13         classification_report, f1_score,
14         ConfusionMatrixDisplay
15     import skfuzzy as fuzz
16     from skfuzzy import control as ctrl
17     import warnings

```

```

14 warnings.filterwarnings("ignore")
15 results = pd.read_csv('data/results.csv')
16 races = pd.read_csv('data/races.csv')
17 drivers = pd.read_csv('data/drivers.csv')
18 constructors = pd.read_csv('data/constructors.csv')
19 qualifying = pd.read_csv('data/qualifying.csv')
20 driver_standings = pd.read_csv('data/
    driver_standings.csv')
21 constructor_standings = pd.read_csv('data/
    constructor_standings.csv')
22
23 print('Rozmiary zbiorów:')
24 print(f'results:      {results.shape}')
25 print(f'races:       {races.shape}')
26 print(f'drivers:      {drivers.shape}')
27 print(f'constructors: {constructors.shape}')
28 print(f'qualifying:   {qualifying.shape}')
29
30 df = results.merge(races[['raceId', 'year', '
    circuitId', 'round']], on='raceId')
31 df = df.merge(drivers[['driverId', 'nationality']],
    on='driverId')
32 df = df.merge(constructors[['constructorId', '
    nationality']], on='constructorId', suffixes=(
    '_driver', '_constructor'))
33 df = df.merge(qualifying[['raceId', 'driverId', '
    position']], rename(columns={'position': '
    grid_quali'}), on=['raceId', 'driverId'], how='
    left')
34 df.replace('\N', np.nan, inplace=True)
35 cols_numeric = ['positionOrder', 'grid', 'laps', '
    points', 'milliseconds', 'fastestLapSpeed', '
    grid_quali']
36 for col in cols_numeric:
37     df[col] = pd.to_numeric(df[col], errors='coerce')
38     # Usunięcie wierszy bez pozycji końcowej
39     df = df.dropna(subset=['positionOrder'])
40     print(df)
41     print("=====")
42     # 3 klasy: Podium (1-3), Punkty (4-10), Poza
        punktami (>=11)

```



```

43     def classify_result(pos):
44         if pos <= 3:
45             return 'Podium'
46         elif pos <= 10:
47             return 'Punkty'
48         else:
49             return 'Poza punktami'
50
51     df['target'] = df['positionOrder'].apply(
52         classify_result)
53     print(df['target'].value_counts())
54
55     # Uzupełnienie brakujących wartości
56     df['grid_quali'] = df['grid_quali'].fillna(df['grid
57         ''])
58     df = df.sort_values(['year', 'round'])
59     df['avg_grid_season'] = df.groupby(['year', '
60         driverId'])['grid'].transform(
61         lambda x: x.expanding().mean().shift(1)
62     )
63     df['avg_position_season'] = df.groupby(['year', '
64         driverId'])['positionOrder'].transform(
65         lambda x: x.expanding().mean().shift(1)
66     )
67     df['avg_constructor_season'] = df.groupby(['year',
68         'constructorId'])['positionOrder'].transform(
69         lambda x: x.expanding().mean().shift(1)
70     )
71     # Pozycja w mistrzostwach przed wyścigiem
72     ds = driver_standings.merge(races[['raceId', 'year',
73         'round']], on='raceId')
74     ds = ds.sort_values(['driverId', 'year', 'round'])
75     ds['driver_pos_before_race'] = ds.groupby(['
76         driverId', 'year'])['position'].shift(1).fillna
77         (20)
78     df = df.merge(ds[['raceId', 'driverId', '
79         driver_pos_before_race']], on=['raceId', '
80         driverId'], how='left')
81     df['driver_pos_before_race'] = df['
82         driver_pos_before_race'].fillna(20)

```

```

73 # Pozycja konstruktora w mistrzostwach przed wyś
    cigiem
74 cs = constructor_standings.merge(races[['raceId', '
    year', 'round']], on='raceId')
75 cs = cs.sort_values(['constructorId', 'year', '
    round'])
76 cs['constructor_pos_before_race'] = cs.groupby(['
    constructorId', 'year'])['position'].shift(1).
    fillna(10)
77 df = df.merge(cs[['raceId', 'constructorId', '
    constructor_pos_before_race']], on=['raceId', '
    constructorId'], how='left')
78 df['constructor_pos_before_race'] = df['
    constructor_pos_before_race'].fillna(10)
79
80 # Historia kierowcy na danym torze
81 df = df.sort_values(['driverId', 'circuitId', 'year
    '])
82 df['hist_track_avg'] = df.groupby(['driverId', '
    circuitId'])['positionOrder'].transform(
83 lambda x: x.expanding().mean().shift(1)
84 )
85 df['hist_track_avg'] = df['hist_track_avg'].fillna
    (15)
86 df = df.sort_values(['year', 'round'])
87
88 # Wybór cech do modelu
89 FEATURES = [
90 'grid',
91 'grid_quali',
92 'avg_grid_season',
93 'avg_position_season',
94 'avg_constructor_season',
95 'driver_pos_before_race',
96 'constructor_pos_before_race',
97 'hist_track_avg',
98 ]
99 X = df[FEATURES].copy()
100 y = df['target'].copy()
101 print(X)
102

```

```

103     # Usunięcie wierszy z NaN w cechach
104     mask = X.notna().all(axis=1)
105     X, y = X[mask], y[mask]
106     print(f'Zbiór: {X.shape[0]} próbek, {X.shape[1]}
        cech')
107
108     # Podział na zbiory
109     X_train, X_test, y_train, y_test = train_test_split
        (X, y, test_size=0.2, random_state=42, stratify=
        y)
110
111     # Skalowanie wartości dla SVM i KNN
112     scaler = StandardScaler()
113     X_train_sc = scaler.fit_transform(X_train)
114     X_test_sc = scaler.transform(X_test)
115     print(f'Trening: {X_train.shape[0]} | Test: {X_test
        .shape[0]}')
116
117     # Wizualizacja rozkładu klas
118     fig, axes = plt.subplots(1, 2, figsize=(12, 4))
119     y.value_counts().plot(kind='bar', ax=axes[0], color
        =['gold', 'steelblue', 'gray'], edgecolor='black
        ')
120     axes[0].set_title('Rozkład klas w zbiorze')
121     axes[0].set_xlabel('Klasa')
122     axes[0].set_ylabel('Liczba próbek')
123     axes[0].tick_params(axis='x', rotation=0)
124     X['grid'].hist(ax=axes[1], bins=20, color='
        steelblue', edgecolor='black')
125     axes[1].set_title('Rozkład pozycji startowej (grid)
        ')
126     axes[1].set_xlabel('Pozycja startowa')
127     axes[1].set_ylabel('Liczba')
128     plt.tight_layout()
129     plt.show()
130     iso = IsolationForest(contamination=0.05,
        random_state=42)
131     anomaly_labels = iso.fit_predict(X_train_sc)
132
133     # -1 = anomalia, 1 = normalna obserwacja
134     n_anomalies = (anomaly_labels == -1).sum()

```

```

135     print(f'Wykryte anomalie: {n_anomalies} ({
        n_anomalies/len(anomaly_labels)*100:.1f}% zbioru
        treningowego)')
136     brazil_2024 = df[df['raceId'] == 1141].copy()
137     verstappen = brazil_2024[brazil_2024['driverId'] ==
        830]
138     print(verstappen[['driverId', 'grid', '
        positionOrder', 'points', 'laps', 'target']])
139     def evaluate_model(name, model, X_tr, y_tr, X_te,
        y_te):
140         model.fit(X_tr, y_tr)
141         y_pred = model.predict(X_te)
142         acc = accuracy_score(y_te, y_pred)
143         f1 = f1_score(y_te, y_pred, average='weighted')
144         print(f'\n=== {name} ===')
145         print(f'Accuracy:         {acc:.4f}')
146         print(f'F1 (weighted):    {f1:.4f}')
147         print(classification_report(y_te, y_pred))
148         _, ax = plt.subplots(figsize=(6, 4))
149         ConfusionMatrixDisplay.from_predictions(y_te,
            y_pred, ax=ax, colorbar=False, cmap='Blues')
150         ax.set_title(f'Confusion Matrix - {name}')
151         plt.tight_layout()
152         plt.show()
153         return model, acc, f1
154         rf_base = RandomForestClassifier(n_estimators=100,
            random_state=42, class_weight='balanced')
155         rf_base, rf_acc, rf_f1 = evaluate_model(
156             'Random Forest (bazowy)', rf_base,
157             X_train, y_train,
158             X_test, y_test
159         )
160         param_grid_rf = {
161             'n_estimators': [100, 200, 300],
162             'max_depth': [None, 10, 20],
163             'min_samples_split': [2, 5, 10]
164         }
165         grid_rf = GridSearchCV(
166             RandomForestClassifier(random_state=42,
                class_weight='balanced'),
            param_grid_rf,

```

```

168         cv=5,
169         scoring='f1_macro',
170         n_jobs=-1,
171         verbose=1
172     )
173     grid_rf.fit(X_train, y_train)
174     print(f'\nNajlepsze parametry RF: {grid_rf.
175           best_params_}')
176     rf_best = grid_rf.best_estimator_
177     rf_acc, rf_f1 = evaluate_model(
178         'Random Forest (po kalibracji)', rf_best,
179         X_train, y_train,
180         X_test, y_test
181     )
182     svm_base = SVC(kernel='rbf', random_state=42,
183                    probability=True)
184     svm_base, svm_acc, svm_f1 = evaluate_model(
185         'SVM (bazowy)', svm_base,
186         X_train_sc, y_train,
187         X_test_sc, y_test
188     )
189     param_grid_svm = {
190         'C': [0.1, 1, 10],
191         'gamma': ['scale', 0.01],
192         'kernel': ['rbf']
193     }
194     grid_svm = GridSearchCV(
195         SVC(probability=True, random_state=42, class_weight
196            = 'balanced'),
197         param_grid_svm,
198         cv=5,
199         scoring='f1_macro',
200         n_jobs=-1,
201         verbose=1
202     )
203     grid_svm.fit(X_train_sc, y_train)
204     print(f'\nNajlepsze parametry SVM: {grid_svm.
205           best_params_}')
206     svm_best = grid_svm.best_estimator_
207     svm_acc, svm_f1 = evaluate_model(
208         'SVM (po kalibracji)', svm_best,

```

```

205     X_train_sc, y_train,
206     X_test_sc, y_test
207 )
208 knn_base = KNeighborsClassifier(n_neighbors=5)
209 knn_base, knn_acc, knn_f1 = evaluate_model(
210     'KNN (bazowy)', knn_base,
211     X_train_sc, y_train,
212     X_test_sc, y_test
213 )
214 param_grid_knn = {
215     'n_neighbors': [3, 5, 7, 11, 15],
216     'weights':      ['uniform', 'distance'],
217     'metric':       ['euclidean', 'manhattan', '
                        minkowski']
218 }
219 grid_knn = GridSearchCV(
220     KNeighborsClassifier(),
221     param_grid_knn,
222     cv=5,
223     scoring='f1_macro',
224     n_jobs=-1,
225     verbose=1
226 )
227 grid_knn.fit(X_train_sc, y_train)
228 print(f'\nNajlepsze parametry KNN: {grid_knn.
        best_params_}')
229 knn_best = grid_knn.best_estimator_
230 knn_best, knn_acc, knn_f1 = evaluate_model(
231     'KNN (po kalibracji)', knn_best,
232     X_train_sc, y_train,
233     X_test_sc, y_test
234 )
235 mlp_base = MLPClassifier(
236     hidden_layer_sizes=(128, 64),
237     activation='relu',
238     max_iter=1000,
239     random_state=42,
240 )
241 mlp_base, mlp_acc, mlp_f1 = evaluate_model(
242     'MLP (bazowy)', mlp_base,
243     X_train_sc, y_train,

```

```

244 X_test_sc, y_test
245 )
246 param_grid_mlp = {
247     'hidden_layer_sizes': [(64, 32), (128, 64),
248                             (128, 64, 32)],
249     'activation':          ['relu', 'tanh'],
250     'learning_rate_init': [0.001, 0.01],
251     'alpha':               [0.0001, 0.001]
252 }
253 grid_mlp = GridSearchCV(
254     MLPClassifier(max_iter=2000, random_state=42),
255     param_grid_mlp,
256     cv=5,
257     scoring='f1_macro',
258     n_jobs=-1,
259     verbose=1
260 )
261 grid_mlp.fit(X_train_sc, y_train)
262 print(f'\nNajlepsze parametry MLP: {grid_mlp.
263       best_params_}')
264 mlp_best = grid_mlp.best_estimator_
265 mlp_acc, mlp_f1 = evaluate_model(
266     'MLP (po kalibracji)', mlp_best,
267     X_train_sc, y_train,
268     X_test_sc, y_test
269 )
270 models_summary = {}
271 for name, model in [('RF', rf_best), ('SVM',
272                                       svm_best), ('KNN', knn_best), ('MLP', mlp_best)
273                  ]:
274     X_te = X_test if name == 'RF' else X_test_sc
275     y_pred = model.predict(X_te)
276     models_summary[name] = {
277         'Accuracy': accuracy_score(y_test, y_pred),
278         'F1':       f1_score(y_test, y_pred,
279                               average='weighted')
280     }
281
282 summary_df = pd.DataFrame(models_summary).T
283 print(summary_df.round(4))

```

```

279 summary_df.plot(kind='bar', figsize=(9, 4),
    edgecolor='black', color=['steelblue', 'coral'])
280 plt.title('Porównanie modeli po kalibracji')
281 plt.ylabel('Wartość metryki')
282 plt.ylim(0, 1)
283 plt.xticks(rotation=0)
284 plt.legend()
285 plt.tight_layout()
286 plt.show()
287 rf_best.fit(X_train_sc, y_train)
288 expert_system = VotingClassifier(
289     estimators=[
290         ('rf', rf_best),
291         ('svm', svm_best),
292         ('knn', knn_best),
293         ('mlp', mlp_best)
294     ],
295     voting='soft'
296 )
297 expert_system.fit(X_train_sc, y_train)
298 y_pred_expert = expert_system.predict(X_test_sc)
299 acc_expert = accuracy_score(y_test,
    y_pred_expert)
300 f1_expert = f1_score(y_test, y_pred_expert,
    average='weighted')
301 print(f'=== System Ekspercki (VotingClassifier) ===')
302 print(f'Accuracy: {acc_expert:.4f}')
303 print(f'F1 (weighted): {f1_expert:.4f}')
304 print(classification_report(y_test, y_pred_expert))
305 fig, ax = plt.subplots(figsize=(6, 4))
306 ConfusionMatrixDisplay.from_predictions(
307     y_test, y_pred_expert, ax=ax,
308     colorbar=False, cmap='Greens'
309 )
310 ax.set_title('Confusion Matrix - System Ekspercki')
311 plt.tight_layout()
312 plt.show()
313 zuzycie_opon = ctrl.Antecedent(np.arange(0, 101,
    1), "zuzycie_opon")

```



```

314 przewaga          = ctrl.Antecedent(np.arange(-30,
315           31, 1), "przewaga")
numer_okrazenia     = ctrl.Antecedent(np.arange(0, 71,
316           1), "numer_okrazenia")
predykcja_ml       = ctrl.Antecedent(np.arange(0,
317           3.01, 0.01), "predykcja_ml")
pilnosc_pit        = ctrl.Consequent(np.arange(0, 101,
318           1), "pilnosc_pit")

319 # FUNKCJE PRZYNALEŻNOŚCI - ZUŻYCIE OPON
320 zuzycie_opon["niskie"] = fuzz.trimf(zuzycie_opon.
321   universe, [0, 0, 40])
zuzycie_opon["srednie"] = fuzz.trimf(zuzycie_opon.
322   universe, [20, 50, 80])
zuzycie_opon["wysokie"] = fuzz.trimf(zuzycie_opon.
323   universe, [60, 100, 100])

324 # FUNKCJE PRZYNALEŻNOŚCI - PRZEWAGA
325 przewaga["strata"]     = fuzz.trimf(przewaga.
326   universe, [-30, -30, 0])
przewaga["neutralna"]   = fuzz.trimf(przewaga.
327   universe, [-10, 0, 10])
przewaga["prowadzenie"] = fuzz.trimf(przewaga.
328   universe, [0, 30, 30])

329 # FUNKCJE PRZYNALEŻNOŚCI - NUMER OKRĄŻENIA
330 numer_okrazenia["wczesne"] = fuzz.trimf(
331   numer_okrazenia.universe, [0, 0, 25])
numer_okrazenia["srednie"] = fuzz.trimf(
332   numer_okrazenia.universe, [15, 35, 55])
numer_okrazenia["pozne"]   = fuzz.trimf(
333   numer_okrazenia.universe, [45, 70, 70])

334 # FUNKCJE PRZYNALEŻNOŚCI - PREDYKCJA ML
335 # 0 = Poza punktami, 1 = Punkty, 2 = Podium
336 predykcja_ml["slaby"]    = fuzz.trimf(predykcja_ml.
337   universe, [0, 0, 1])
predykcja_ml["sredni"]    = fuzz.trimf(predykcja_ml.
338   universe, [0.5, 1, 1.5])
predykcja_ml["mocny"]     = fuzz.trimf(predykcja_ml.
   universe, [1, 2, 3])

```

```

339
340 # FUNKCJE PRZYNALEŻNOŚCI - PILNOŚĆ PIT-STOPU
341 pilnosc_pit["niska"] = fuzz.trimf(pilnosc_pit.
    universe, [0, 0, 40])
342 pilnosc_pit["srednia"] = fuzz.trimf(pilnosc_pit.
    universe, [20, 50, 80])
343 pilnosc_pit["wysoka"] = fuzz.trimf(pilnosc_pit.
    universe, [60, 100, 100])
344
345 fig, axes = plt.subplots(3, 2, figsize=(13, 12))
346
347 zmienne = [
348     (zuzycie_opon, 'Zużycie opon',
349         axes[0, 0]),
350     (przewaga, 'Przewaga nad rywalem [s]',
351         axes[0, 1]),
352     (numer_okrazenia, 'Numer okrążeń',
353         axes[1, 0]),
354     (predykcja_ml, 'Predykcja ML (0=Poza pkt, 1=
355         Punkty, 2=Podium)', axes[1, 1]),
356     (pilnosc_pit, 'Pilność pit-stopu (wyjście)',
357         axes[2, 0]),
358 ]
359
360 for zmienna, tytul, ax in zmienne:
361     for nazwa, mf in zmienna.terms.items():
362         ax.plot(zmienna.universe, mf.mf, label=nazwa)
363     ax.set_title(tytul)
364     ax.set_ylim(0, 1.1)
365     ax.legend()
366     ax.set_xlabel(zmienna.label)
367     ax.set_ylabel('Membership')
368
369 axes[2, 1].axis('off')
370 plt.suptitle('Funkcje przynależności', fontsize=13)
371 plt.tight_layout()
372 plt.show()
373
374 # MAPOWANIE PREDYKCJI ML NA LICZBĘ
375 # System rozmyty operuje na liczbach, więc mapujemy
    klasy

```

```

370     klasa_na_liczbe = {"Poza punktami": 0, "Punkty": 1,
371                        "Podium": 2}
372
373     rules = [
374         # Reguła 1: Opony bardzo zużyte -> pit
375         ctrl.Rule(zuzycie_opon["wysokie"], pilnosc_pit["
376             wysoka"]),
377         # Reguła 2: Opony świeże + 1-miejsce -> zostań
378         ctrl.Rule(zuzycie_opon["niskie"] & przewaga["
379             prowadzenie"], pilnosc_pit["niska"]),
380         # Reguła 3: Środkowe okrążenia + średnie zużycie ->
381         być może pit
382         ctrl.Rule(numer_okrazenia["srednie"] & zuzycie_opon
383             ["srednie"], pilnosc_pit["srednia"]),
384         # Reguła 4: Poza punktami + późne okrążenia -> pit
385         ctrl.Rule(przewaga["strata"] & numer_okrazenia["
386             pozne"], pilnosc_pit["wysoka"]),
387         # Reguła 5: Wczesne okrążenia -> nie pit
388         ctrl.Rule(numer_okrazenia["wczesne"], pilnosc_pit["
389             niska"]),
390         # Reguła 6: Średnie zużycie + strata -> rozważ pit
391         ctrl.Rule(zuzycie_opon["srednie"] & przewaga["
392             strata"], pilnosc_pit["srednia"]),
393         # Reguła 7: Prowadzenie + późne okrążenia + średnie
394         zużycie -> być może pit
395         ctrl.Rule(przewaga["prowadzenie"] & numer_okrazenia
396             ["pozne"] & zuzycie_opon["srednie"], pilnosc_pit
397             ["srednia"]),
398         # Reguła 8: Modele przewidują słaby wynik + średnie
399         zużycie -> pit
400         ctrl.Rule(predykcja_ml["slaby"] & zuzycie_opon["
401             srednie"], pilnosc_pit["wysoka"]),
402         # Reguła 9: Modele przewidują podium + prowadzenie
403         -> nie pit
404         ctrl.Rule(predykcja_ml["mocny"] & przewaga["
405             prowadzenie"], pilnosc_pit["niska"]),
406         # Reguła 10: Modele przewidują podium + średnie zuż
407         ycie -> być może pit
408         ctrl.Rule(predykcja_ml["mocny"] & zuzycie_opon["
409             srednie"], pilnosc_pit["niska"]),

```

```

394 # Reguła 11: Modele niepewne (Punkty) + wysokie zuż
      ycie -> pit
395 ctrl.Rule(predykcja_ml["sredni"] & zuzycie_opon["
      wysokie"], pilnosc_pit["wysoka"]),
396 # Reguła 12: Modele niepewne (Punkty) + niskie zuż
      ycie -> nie pit
397 ctrl.Rule(predykcja_ml["sredni"] & zuzycie_opon["
      niskie"], pilnosc_pit["niska"]),
398 ]
399 # SILNIK WNIOSKOWANIA
400 pit_ctrl = ctrl.ControlSystem(rules)
401 pit_sim = ctrl.ControlSystemSimulation(pit_ctrl)
402
403 # TEST Z PREDYKCJĄ SYSTEMU EKSPERCKIEGO
404 # Pobieramy predykcję VotingClassifier dla przykła
      dowego wiersza z danych testowych
405 przyklad = X_test_sc[0].reshape(1, -1)
406 predykcja_str = expert_system.predict(przyklad)[0]
407 predykcja_num = klasa_na_liczbe[predykcja_str]
408 print(f"Predykcja systemu eksperckiego: {
      predykcja_str}")
409
410 # Scenariusz: okrążenie 40/70, opony 75% zużyte, 5s
      za rywalem
411 pit_sim.input["zuzycie_opon"] = 75
412 pit_sim.input["przewaga"] = -5
413 pit_sim.input["numer_okrazenia"] = 40
414 pit_sim.input["predykcja_ml"] = predykcja_num
415 pit_sim.compute()
416
417 wynik = pit_sim.output["pilnosc_pit"]
418 print(f"Pilność pit-stopu: {wynik:.1f}/100")
419 if wynik >= 65:
420 rekomendacja = "PIT"
421 elif wynik >= 35:
422 rekomendacja = "Rozważ pit-stop"
423 else:
424 rekomendacja = "Zostań na torze"
425 print(f"Rekomendacja: {rekomendacja}")
426

```

```
427         # Wizualizacja wyniku rozmytego (zagregowany obszar
          aktywne)
428     pilnosc_pit.view(sim=pit_sim)
429     plt.title(f'Wynik systemu rozmytego - pilność pit-
          stopu: {wynik:.1f}%')
430     plt.show()
```